UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR
ÉCOLE SUPÉRIEURE POLYTECHNIQUE (ESP)
Département de Génie Informatique

TRAITEMENT NUMÉRIQUE DU SIGNAL

DIC 2 – 4ème année – Génie Informatique

MINI-PROJET

Numérisation, Segmentation et Filtrage d'un Signal Vocal

Application web avec Python & Flask

Enseignant responsable :	Dr. Moustapha MBAYE
Niveau :	DIC 2 – 4ème année
Modalité :	Travail en binôme
Année universitaire :	2025–2026

DOCUMENT DISTRIBUÉ AUX ÉTUDIANTS

À lire intégralement avant de commencer

1. Sujet du Mini-Projet

Réaliser en **Python** une application web avec **Flask** comportant deux parties complémentaires portant sur le traitement d'un signal vocal : la numérisation et segmentation (Partie 1), et l'analyse fréquentielle avec filtrage (Partie 2).

2. Partie 1 – Numérisation et Segmentation de la Voix

2.1 Paramètres de Numérisation

L'application doit permettre à l'utilisateur de choisir, *avant tout enregistrement* :

Paramètre	Valeurs autorisées
Fréquence d'échantillonnage	16 kHz 22,05 kHz 44,1 kHz
Codage	16 bits 32 bits
Format de sortie	WAV uniquement
Durée d'enregistrement	définie librement par l'utilisateur (en secondes)

Contrainte

Seules les valeurs du tableau ci-dessus sont autorisées. Toute autre valeur doit être rejetée avec un message d'erreur.

2.2 Enregistrement et Base de Données Audio

L'utilisateur doit pouvoir enregistrer sa voix depuis le navigateur et sauvegarder le fichier `.wav` dans une **base de données locale** organisée :

```
database/
|-- locuteur_01/
|   |-- session_01/
|   |   |-- enreg_001_16kHz_16b.wav
|   |   `-- enreg_002_44kHz_32b.wav
|   `-- session_02/
`-- locuteur_02/
    `-- ...
```

2.3 Segmentation Automatique

L'application doit détecter automatiquement les silences et découper le fichier audio en **segments vocaux utiles**. L'utilisateur configure :

- le **seuil d'amplitude** (niveau en dessous duquel c'est un silence) ;
- la **durée minimale de silence** (en millisecondes).

Les segments obtenus sont sauvegardés dans un dossier `segments/` et listés dans l'interface avec des liens de lecture et de téléchargement.

3. Partie 2 – Analyse Fréquentielle et Filtrage

3.1 Chargement et Affichage

1. Charger un fichier audio (WAV, MP3, OGG, etc.) depuis l'interface.
2. Si le fichier n'est **pas en WAV**, le convertir automatiquement.
3. Afficher le **signal temporel** (amplitude en fonction du temps).
4. Calculer et afficher le **spectre d'amplitude par FFT** (axe x : fréquences en Hz, axe y : amplitudes).

3.2 Identification Visuelle du Bruit

À partir du spectre affiché, **identifier manuellement** les zones fréquentielles qui semblent correspondre au bruit. Cette étape est volontairement visuelle : vous devez être capables d'expliquer votre choix à partir du spectre.

3.3 Définition du Filtre

Contrainte ABSOLUE de filtrage

Le filtrage doit être réalisé **exclusivement** par un masque fréquentiel rectangulaire, selon l'un des deux types suivants :

- **Filtre passe-bande :** on *conserve* les fréquences dans $[f_{\min}, f_{\max}]$
- **Filtre coupe-bande :** on *supprime* les fréquences dans $[f_{\min}, f_{\max}]$

Tout autre filtre (Butterworth, Hanning, Hamming...) entraîne **un zéro sur cette partie**.

Formulation mathématique du masque :

$$H(f) = \begin{cases} 1 & \text{si } f_{\min} \leq |f| \leq f_{\max} \quad \text{(passe-bande)} \\ 0 & \text{sinon} \end{cases} \qquad \text{ou} \quad \bar{H}(f) = 1 - H(f) \quad \text{(coupe-bande)}$$

L'utilisateur saisit $f_{\min}$ et $f_{\max}$ en Hz, et choisit le type.

3.4 Application du Filtre et Restitution

Une fois le filtre défini, l'application doit :

1. appliquer le masque au spectre FFT ;
2. reconstruire le signal par IFFT ;
3. afficher les spectres et signaux **avant/après filtrage** ;
4. permettre l'**écoute** du signal filtré dans le navigateur ;
5. permettre le **téléchargement** du fichier WAV filtré.

4. Structure des Interfaces Flask

4.1 Interface 1 – Numérisation / Segmentation

Élément	Description
Sélecteur de fréquence	Liste déroulante : 16 kHz / 22,05 kHz / 44,1 kHz
Codage	Boutons radio : 16 bits / 32 bits
Durée	Champ numérique (secondes)
Bouton Enregistrer	Lance / arrête la capture microphone
Bouton Sauvegarder	Enregistre le fichier `.wav`
Seuil + Durée silence	Paramètres de segmentation
Bouton Segmenter	Lance le découpage
Liste des segments	Tableau avec lecture et téléchargement

4.2 Interface 2 – FFT / Filtrage

Élément	Description
Upload fichier	Chargement + conversion auto en WAV
Graphique temporel	Signal $x(t)$
Graphique FFT	Spectre $\lvert X(f)\rvert$
$f_{\min}$ **et** $f_{\max}$	Bornes du filtre en Hz
Type de filtre	Radio : passe-bande / coupe-bande
Bouton Filtrer	Applique le masque et reconstruit
Comparaison avant/après	Graphiques superposés
Lecteur HTML5	Écoute directe du signal filtré
Téléchargement	Export du fichier WAV filtré

5. Contraintes Techniques

- **Langage :** Python 3.x uniquement
- **Framework web :** Flask uniquement
- **Format audio :** WAV imposé (conversion automatique si besoin)
- **Filtrage :** rectangle ou rectangle opposé **uniquement**
- **Interfaces :** deux interfaces distinctes (Partie 1 et Partie 2)
- **Code :** chaque fonction doit être commentée (docstring)
- **Travail :** en binôme obligatoirement

Bibliothèques Python recommandées :

Package	Rôle
`flask`	Serveur web et routage
`numpy`	Calculs numériques et FFT
`scipy.fft`	Transformée de Fourier Rapide
`scipy.io.wavfile`	Lecture / écriture WAV
`librosa`	Segmentation par silence
`sounddevice`	Enregistrement microphone
`matplotlib`	Génération des graphiques
`pydub`	Conversion de formats audio

6. Livrables à Remettre

1. **Code source complet** de l'application, commenté et structuré
2. **Application Flask fonctionnelle**, testable en local
3. **Rapport PDF** contenant :
   - architecture et organisation du code ;
   - justification des paramètres de numérisation choisis ;
   - analyse commentée des spectres avant/après filtrage ;
   - explication du choix des bornes fréquentielles ;
   - difficultés rencontrées et solutions apportées.
4. **Vidéo de démonstration** (3–5 min) montrant :
   - un enregistrement vocal avec paramètres choisis ;
   - la segmentation automatique ;
   - l'identification visuelle du bruit sur le spectre FFT ;
   - l'application du filtre et l'écoute du résultat filtré.

7. Grille d'Évaluation

Critère	Points
Partie 1 – Numérisation / Segmentation	**40**
Paramètres de numérisation corrects	10
Enregistrement et sauvegarde WAV	10
Segmentation automatique fonctionnelle	10
Organisation de la base de données	10
Partie 2 – FFT / Filtrage	**40**
Chargement et conversion	5
Affichage du spectre FFT	10
Filtre rectangulaire correctement appliqué	15
Reconstruction et export du signal	10
Qualité du code et documentation	**10**
Commentaires, lisibilité, structure	5
Rapport + vidéo de démonstration	5
TOTAL	**90**
Bonus – originalité de l'interface	*+10*

8. Consignes Importantes

1. **Binôme obligatoire.** Toute soumission individuelle est refusée.
2. **Commenter le code.** Chaque fonction doit avoir une docstring (rôle, paramètres, valeur de retour).
3. **Justifier les paramètres.** Expliquer dans le rapport et dans la vidéo pourquoi vous avez choisi telle fréquence d'échantillonnage, tel seuil, telles bornes de filtre.
4. **Analyser le spectre dans la vidéo.** Montrez le graphique FFT et expliquez *visuellement et à voix haute* pourquoi vous avez sélectionné cette bande fréquentielle.
5. **Tester** sur au moins deux fichiers audio différents.

Tout plagiat de code entre binômes, ou copie depuis Internet sans adaptation et référence explicit, entraîne la note **zéro** pour les deux membres et peut donner lieu à des sanctions disciplinaires.

9. Références Utiles

- **Flask :** `https://flask.palletsprojects.com`
- **SciPy FFT :** `https://docs.scipy.org/doc/scipy/reference/fft.html`
- **LibROSA :** `https://librosa.org/doc/latest/index.html`

- **SoundDevice :** https://python-sounddevice.readthedocs.io
- **Cours TNS – Dr. Moustapha MBAYE :** polycopiés distribués en cours
- A. V. Oppenheim & R. W. Schafer, *Discrete-Time Signal Processing*, Pearson, 2010.